

Internship Tracker & Auto-Apply Assistant

Complete Technical Documentation — Phase 1

Production: <https://web-production-64a44.up.railway.app>

Repository: github.com/joshuamichael365/internship-tracker (private)

Owner: Joshua Michael (joshuamichael365@gmail.com) — single-user application

Document generated: July 2, 2026 · Built with Claude Code (Fable 5)

This document is self-contained: it explains the purpose, architecture, technology choices, data model, every subsystem, the security model, deployment, and the roadmap in enough detail that a person — or another AI assistant — can fully understand how the application works without access to the codebase.

1. What This Application Is

A personal, single-user web application that continuously monitors the internet for new Software Engineering / Machine Learning / CS internship postings, notifies the owner within about a minute of a posting appearing in a tracked source, organizes postings for browsing and tracking, and assists with applying — drafting cover letters and short-answer responses in the owner's own writing voice and auto-filling application forms through a companion Chrome extension. The owner explicitly chooses, per application, how much automation to allow (Manual / Agentic Assist / Full Auto-Apply).

Design language is Apple Human Interface Guidelines adapted for web: SF Pro system font stack, sidebar navigation, rounded cards, soft shadows, light and dark mode, calm information-dense layouts.

Phase status

- **Phase 1 (SHIPPED, this document):** discovery engine, all notification channels, full tracker, Agentic Assist (Mode 2) with drafting + document storage + extension autofill, Manual mode, profile/resume management, deployed 24/7 on Railway.
- **Phase 2 (next):** Full Auto-Apply (Mode 3) with per-application opt-in and pre-submit preview, blocker retry/notify, Gmail status monitoring (opt-in, read-only), interview/skill prep recommendations.
- **Phase 3:** visa-sponsorship filter UI (data already collected), analytics dashboard, data export, smart-ranked digests.

The three automation modes (core product concept)

Mode	What the app does	Status
Manual	Just tracks the application; owner fills the portal themselves.	Shipped
Agentic Assist	Drafts cover letter + short answers in the owner's voice (always editable), saves documents to chosen storage, extension fills the portal form — but the owner reviews everything and clicks Submit personally. The app never submits.	Shipped
Full Auto-Apply	Same as Assist, plus the app submits — only after a deliberate per-application opt-in showing exactly what will be sent. Never a global default.	Phase 2

Non-negotiable rules enforced in code: the mode is NULL until the owner picks one (never defaulted); a recommendation engine suggests a mode but the owner's choice always wins and overrides are logged so the recommender learns; every AI draft is editable before use; the mode badge is visible everywhere an application appears.

2. System Architecture

The system is a pnpm-workspaces monorepo with three deployable apps and two shared packages, deployed as three Railway services (web, worker, Postgres).

```
internship-tracker/
  apps/web           Next.js 16 (App Router) - UI, API routes, server actions, PWA service worker
  apps/worker        Node service - pollers, ingest pipeline, notification dispatch, cron jobs
  apps/extension      Chrome MV3 extension - form autofill in the owner's own browser
  packages/db         Drizzle ORM schema + migrations (PostgreSQL), re-exports query operators
  packages/shared     Types + pure logic: normalization, dedupe hashing, tagging, rule evaluation
```

Data flow (discovery to notification)

```
[GitHub repos / Greenhouse / Lever / SmartRecruiters / Workday / RSS]
|  polled every 60s by graphile-worker cron (conditional requests: ETag/If-Modified-Since)
v
normalize -> dedupe (FNV-1a hash of company|title|location) -> tag (role/level/mode/terms)
|          duplicates merge into one card, recording every source that carried it
v
suppress if owner already applied to same company+role -> store in postings table
|
v
notification rules (role types, work modes, excluded companies, keywords, new-grad gate)
|
+-- inside quiet hours (default 11 PM - 7 AM ET) -> digest_queue -> single morning digest
+-- otherwise -> instant fan-out: web push + email (+ SMS if enabled) + notification_log
```

Why polling and not webhooks: none of the sources offer webhooks/streams to outside consumers. 60-second conditional polling costs almost nothing (most ticks are HTTP 304) and achieves ~1-minute source-to-phone latency, which meets the real-time goal in practice.

Why the form-filler is a browser extension (key decision)

Mode 2 autofill runs in the owner's own Chrome via an extension rather than a server-side headless browser. Rationale: it uses the owner's real logged-in portal sessions (no credential storage at all in Phase 1), it is immune to the bot-detection arms race that breaks headless automation on ATS portals like Workday, and the owner literally watches fields fill and can correct them before submitting. Server-side automation is deferred to Phase 2, only for Full Auto-Apply, and only if a reliability track record justifies it.

Cross-service job queue

The web app and worker never call each other over HTTP. They share the Postgres database, and the web app enqueues work for the worker with graphile-worker's SQL function, e.g. marking an application Applied runs: `select graphile_worker.add_job('send_confirmation', ...)`. graphile-worker (Postgres-backed) was chosen over Redis-based queues to avoid paying for and operating a second datastore.

3. Technology Stack and Why Each Piece

Layer	Choice	Why
Framework	Next.js 16 (App Router), React 19, TypeScript	One codebase for UI + API routes + server actions; static/dynamic rendering per page; PWA service worker for push.
Styling	Tailwind CSS v4 with custom HIG token theme	Apple-like design tokens (colors, radii, shadows) defined once as CSS variables, light/dark via class strategy.
Database	PostgreSQL 17 + Drizzle ORM	Typed schema in TypeScript, SQL-first migrations (drizzle-kit), jsonb for flexible fields (locations, terms, rules, drafts).
Background jobs	graphile-worker	Cron + queue on top of Postgres itself - no Redis to pay for; exactly-once job semantics; SQL enqueue from the web app.
Auth	Auth.js v5 (next-auth beta.31), Google provider, JWT sessions	Single user: email hard-allowlist in the signIn callback; JWT sessions mean no session tables; AUTH_URL + AUTH_TRUST_HOST for Railway's proxy.
AI	Anthropic API - claude-sonnet-5 (drafting), claude-haiku-4-5 (resume parsing)	Sonnet for quality prose in the owner's voice (~2-4 cents/letter); Haiku for cheap structured extraction. Posting tagging uses free keyword heuristics.
Web push	web-push (VAPID) + hand-written service worker	Standard Web Push works on desktop + iOS (via Home Screen app); no Firebase dependency.
Email	Resend REST API via fetch	Free tier (100/day), no SDK needed for one endpoint; sends from onboarding@resend.dev.
SMS	Twilio REST API via fetch	Built and wired, but OFF by default - explicit opt-in toggle in Settings; requires A2P 10DLC registration (~\$5-10/mo) when enabled.
PDF generation	pdf-lib	Pure-JS PDF creation for cover letters / short answers; no native deps.
Feeds	fast-xml-parser	RSS/Atom parsing for generic feeds and Instagram-mirror bridges.
Extension	Chrome Manifest V3, vanilla JS, no build step	activeTab + scripting permissions only - it can act ONLY on the tab the owner clicks 'Fill' on; loads unpacked.
Hosting	Railway: web + worker + Postgres, volume at /data	Always-on 24/7 for ~\$5-10/mo; deploys from GitHub main automatically; Railpack builder with per-service RAILPACK_BUILD_CMD/START_CMD variables.

4. Data Model (PostgreSQL, all tables)

Table	Purpose / notable columns
sources	Owner-managed discovery sources. kind (github_repo greenhouse lever smartrecruiters workday rss instagram_mirror), config jsonb (repo/board/feed details), enabled, http_cache (stored ETag/Last-Modified for conditional requests), last_error surfaces polling failures in the UI.
postings	One card per unique internship. dedupe_hash (unique, FNV-1a 64-bit of normalized company title first location), role_type (swe ml data quant other), job_level (internship new_grad), location_mode, terms jsonb ([{"Summer 2026"}]), sponsorship (sponsors citizens_only unknown), deadline, status (active expired hidden), bookmarked, notes, seen_in jsonb (every source that carried it), raw jsonb (original source record).
applications	Tracked applications. stage (saved in_progress applied assessment interviewing offer rejected), mode (manual assist auto - NULL until explicitly chosen), mode_recommendation jsonb (recommended mode + human-readable reasons + signals), drafts jsonb (the reviewed cover letter + Q&A; text the extension fills), resume_id, applied_at, url, notes.
reminders	Follow-ups and OA deadlines per application: label, due_at, done, notified_at (worker fires a notification when due).
resumes	Multiple resume versions: file_key on disk/volume, parsed jsonb (Claude-extracted education/experience/skills/projects), is_default.
writing_samples	Two distinct sets (set = cover_letter short_answer) because the owner writes differently in each; used as few-shot voice references.
documents	Generated files: kind (cover_letter short_answers resume), destination (inapp local gdrive), location = disk key under /Internships//_/.
profile	Single-row structured auto-fill data (name, contact, education, links, work auth).
settings	Single row: timezone (America/New_York), quiet hours (23-7 default), channels jsonb {push, email, sms:false}, include_new_grad, notification_rules jsonb, storage_destination.
push_subscriptions	Web Push endpoints per device; expired ones (HTTP 404/410) are auto-pruned on send.
notification_log	Audit of every delivered notification: channel, kind (instant digest confirmation blocker), payload.
digest_queue	Postings that matched during quiet hours; flushed as one morning digest, then stamped sent_at.
mode_decisions	Recommendation-vs-choice history (recommended, chosen, signals) - the learning data for the recommendation engine.

5. Subsystem Deep-Dives

5.1 Discovery engine

- **GitHub repo poller:** for repos like SimplifyJobs/Summer2026-Internships and vanshb03/Summer2027-Internships it fetches the machine-readable .github/scripts/listings.json (falls back to parsing the README markdown table, including '■' continuation rows). Respects active/is_visible flags; extracts sponsorship and terms.
- **ATS pollers:** Greenhouse boards API, Lever postings API, SmartRecruiters public postings API, Workday CXS JSON endpoint (POST, searchText='intern'). Full company boards are pre-filtered to internship/new-grad-relevant roles.
- **RSS / Instagram mirror:** generic RSS/Atom polling; the instagram_mirror kind exists for RSS-bridge style feeds of public accounts (e.g. zero2sudo). Research finding: Instagram's official API only serves accounts that grant access, X reads are paid-tier, so community repos are the primary real-time signal and mirrors are best-effort supplements. All methods must be ToS-compliant per the spec.
- **Dedupe:** hash of normalized company + title + first location (company suffixes like Inc/LLC stripped, season/year parentheticals stripped). Re-seen postings just append to seen_in - they never re-notify.
- **Tagging:** keyword-heuristic classifiers for role type / level / work mode / terms (verified accurate on ~900 real postings). Claude-assisted tagging is a wired-but-off upgrade path if heuristics ever misclassify.
- **Applied-role suppression:** new postings matching an already-applied company+role arrive with status 'hidden' (visible in Archive, never notified). Marking an application Applied also hides existing active reposts.
- **Auto-archive:** hourly job expires postings whose deadline passed; they move to the Archive view.
- **First-poll backfill:** a brand-new source's first poll ingests silently (no notifications) - adding a 15,000-listing repo does not fire hundreds of alerts.

5.2 Notifications

- **Channels:** web push (service worker shows actions, click deep-links to the posting), email via Resend, SMS via Twilio. Each channel no-ops gracefully with a log line when unconfigured; SMS is additionally gated by the Settings toggle (off by default by owner decision).
- **Rules:** only-these role types, only-these work modes, exclude-companies list, keyword filters, internships-only vs +new-grad - evaluated in packages/shared so web and worker share identical logic.
- **Quiet hours:** timezone-aware (Intl API) window, default 11 PM-7 AM Eastern; matches queue into digest_queue and flush as a single summary the first 5-minute tick after the window ends. Handles windows that wrap midnight.
- **Receipts:** any transition into Applied (manual today, auto-apply in Phase 2) enqueues a confirmation notification - the owner always knows a submission happened.
- **Reminders:** due, un-notified reminders fire through the same channel fan-out every 5 minutes.

5.3 Tracker and the recommendation engine

Kanban board with columns Saved / In Progress / Applied / Assessment-OA / Interviewing / Offer / Rejected. Cards show the mode badge ('Mode not chosen' until picked), reminders due within a week, and move via arrows or a stage select. Applications can be added by pasting any URL - the server fetches the page (with an SSRF guard) and auto-detects role/company from the title, both editable.

The recommendation engine (conservative by design) weighs: ATS familiarity (Greenhouse/Lever/Ashby/SmartRecruiters = 'known', Workday/Taleo/iCIMS = 'hard', anything else = 'unknown'), deadline urgency (<72h tips toward Assist), and the owner's own override history (2+ overrides for the same ATS family flips the recommendation to follow the owner's pattern, and says so). It recommends only Manual or Assist in Phase 1 - Auto-Apply is never recommended until a per-portal reliability track record exists in Phase 2. For hard/unknown portals it says explicitly that it is not confident it can fill the form reliably. Every recommendation-vs-choice pair is stored in mode_decisions.

5.4 Agentic Assist (drafting + documents)

- **Writing samples:** two separate sets (cover letters vs short answers) because the owner's voice differs; up to 3 most-recent samples of the relevant set are injected as few-shot references. More samples can be added anytime and immediately influence future drafts.
- **Drafting:** claude-sonnet-5 with a system prompt that instructs voice-matching and forbids inventing experience not present in the profile. Important caveat: samples are treated as the owner's real track record, so samples must be factually true.
- **Review-first:** drafts land in editable text areas; nothing is used anywhere until the owner saves. Saved raw text is persisted on the application (drafts jsonb) - that exact reviewed text is what the extension fills.
- **Documents:** saved as clean PDFs (pdf-lib) under /Internships//_/CoverLetter.pdf etc. Destination is owner-selected: in-app (server volume; always kept as the retrievable copy), local (immediate browser download), or Google Drive (mirrored into the same folder tree via the Drive API once connected).
- **Offline mode:** before the API key existed, drafting produced clearly-labeled placeholders so the whole flow was testable; the real model activated automatically when the key was configured.

5.5 Chrome extension (the form filler)

Minimal-permission MV3 extension: activeTab + scripting + storage only - no host permissions, no background page, cannot read any page until the owner clicks it. Flow: popup asks the backend /api/assist/packet?url=<current tab> which matches the tab to a tracked application (URL prefix, then hostname) and returns the profile + reviewed drafts + cover-letter PDF URL. 'Fill this page' injects the content script which: fills standard fields by fuzzy label matching (name split into first/last, email, phone, links, education, GPA, work auth), matches drafted answers to portal questions by keyword overlap ($\geq 40\%$ threshold), attaches the cover-letter PDF to file inputs labeled 'cover letter' (DataTransfer API), outlines filled fields green and uncertain/required-but-unfilled fields orange, and reports counts. It never clicks submit buttons - Mode 2's hard stop.

6. Security Model and Audit Results

Authentication and authorization

- Google OAuth sign-in (Auth.js v5), JWT sessions, and a hard allowlist: the signIn callback rejects any Google account whose email is not ALLOWED_EMAIL. Single user by construction.
- Next.js proxy (middleware) protects every route except /api/auth, /signin, static assets, and /api/assist (which uses its own token auth).
- AUTH_DISABLED dev bypass is code-gated to non-production (NODE_ENV check) and is not set anywhere in production.

Extension API security

- Bearer token (EXTENSION_TOKEN, 48 hex chars, random) compared in constant time (crypto.timingSafeEqual); deny-by-default when the variable is unset.
- CORS on /api/assist/* answers preflight and allows only GET with the authorization header; safe to allow any origin because possession of the token is the credential.

Other hardening

- SSRF guard: the paste-a-link fetch only follows public http/https URLs - localhost, RFC-1918 ranges (10/8, 172.16/12, 192.168/16), link-local, and .internal/.local hosts are refused.
- Document downloads resolve only server-generated storage keys from the database - no user-supplied paths touch the filesystem.
- Web push subscriptions and document downloads sit behind the session; expired push endpoints are pruned automatically.
- Automation ethics per spec: extension acts only on explicit click; pollers use conditional requests and a descriptive User-Agent; Phase 2 blocker handling is spec'd as 2-3 retries with backoff, then notify - never hammering.

Secrets handling and audit findings (July 2, 2026)

- Git history scanned: zero secrets ever committed; only .env.example (placeholders) is tracked; data/ and .env* are gitignored. Repository is PRIVATE.
- Production secrets live only in Railway service variables. Dev and prod use different AUTH_SECRET, VAPID key pairs, and EXTENSION_TOKEN.
- Known low-risk event: during Railway setup the CLI echoed the Anthropic API key into the local session transcript on the owner's machine (not git, not public). Not an exposure to third parties; rotating the key in the Anthropic console is optional cheap insurance. No other secret appeared anywhere.
- What the app deliberately does NOT store in Phase 1: no portal passwords (extension uses the owner's own browser sessions), no email access (Gmail monitoring is a Phase 2 opt-in with read-only scope), no third-party user data (single-user).

7. Deployment and Operations

Railway project 'internship-tracker', three services, deploys automatically from GitHub main:

Service	Build / start	Key variables
web	RAILPACK_BUILD_CMD: pnpm --filter web build START: pnpm --filter web start Volume mounted at /data (uploads + generated documents survive deploys)	DATABASE_URL (ref to Postgres), AUTH_SECRET, AUTH_URL, AUTH_TRUST_HOST, AUTH_GOOGLE_ID/SECRET, ALLOWED_EMAIL, VAPID_PUBLIC_KEY, EXTENSION_TOKEN, ANTHROPIC_API_KEY, UPLOAD_DIR=/data/uploads, APP_URL
worker	BUILD: pnpm --filter worker build (typecheck) START: pnpm --filter worker start (tsx)	DATABASE_URL, VAPID_PUBLIC/PRIVATE_KEY, NOTIFY_EMAIL_TO, RESEND_API_KEY, ANTHROPIC_API_KEY, APP_URL, (TWILIO_* when SMS is enabled)
Postgres	Managed Railway Postgres; internal URL for services, public URL used once for migrations/seeding	-

Gotcha worth remembering: behind Railway's proxy, Auth.js needed an explicit AUTH_URL - without it, OAuth callback URLs were generated against the container's internal localhost:8080.

Monthly cost: Railway ~\$5-10 + Anthropic ~\$3-5. Resend/push/volume free. Twilio (~\$5-10) only if the owner enables SMS. Total today: roughly \$10-15/mo.

Local development: Homebrew PostgreSQL 17, pnpm 10 (standalone install in ~/Library/pnpm), 'pnpm dev' runs web (:3000) + worker; env in apps/web/.env.local and root .env (both gitignored).

8. What Was Verified (evidence-based)

- Discovery: first live poll ingested 775 real postings in 0.6s (47 cross-source duplicates merged, 429 irrelevant filtered); second poll was a 105ms HTTP 304. After adding the Summer-2027 repo: 887 active postings; production worker re-verified the same on Railway.
- Auto-archive: synthetic past-deadline posting expired by the hourly job and appeared in Archive.
- Notifications: instant path, quiet-hours queueing, and morning digest flush all exercised; reminder job fired; confirmation receipt flowed web -> SQL enqueue -> worker -> channel fan-out.
- Tracker: Applied transition stamped the date, queued the receipt, and hid the matching repost - all confirmed in the database. Add-by-URL auto-detected a real Greenhouse posting's title.
- Assist: real claude-sonnet-5 cover letter generated in the owner's sample voice; PDFs landed in /Internships/2026/Lambda_2026 Machine Learning Research Intern/; drafts persisted; extension packet API returned them (401 without token; CORS preflight 204).
- Recommendation engine: correctly identified an Ashby portal, recommended Assist with reasons, logged the decision.
- Auth: unauthenticated production requests 307-redirect to /signin with correct callback URL.
- Filters: Summer+2026+ML narrowed 789 -> 91; citizens-only returned exactly the 41 tagged rows.

9. Roadmap

- **Phase 2:** Full Auto-Apply with per-application opt-in + pre-submit preview + limited blocker retries then notify; decide extension-driven vs server-side execution using Phase 1 reliability data; Gmail read-only status monitoring (opt-in, revocable, full-body scope approved by owner, stores only extracted status); per-portal

encrypted credential vault ONLY if server-side automation is chosen; interview/skill-prep recommendations tied to tracked applications.

- **Phase 3:** visa-sponsorship filter UI (column already populated), analytics dashboard (applications/week, response rates, time-to-response), one-click export (CSV + documents zip), smart-ranked digests.
- **Maintenance notes:** add SimplifyJobs' Summer-2027 repo when it opens (one click in Settings); swap-in Haiku tagging if heuristics misclassify; rotate the Anthropic key at leisure; Resend free tier caps at 100 emails/day.

End of document. For the operational runbook see DEPLOYMENT.md in the repository root; for the original product specification and locked decisions, see the plan file referenced in the repo's first commit message.